The vibes that I summoned…

Thoughts on working with AI agents.

Chris Pahl · 2026

Speaker notes — preceding slide

- Welcome the audience. Briefly say what this is NOT: a dunk on AI.
  - It's about finding the right place on the spectrum and not handing over more control than we mean to.
  - ~30 min, plenty of room for discussion after.

The Spectrum (on Reddit)

Full manual
every keystroke yours

Full vibecode
just make it work

⟵ more control · more understanding | more productivity · more delegation ⟶

General tendency:

Using GenAI is a tradeoff between control and productivity.

Speaker notes — preceding slide

- I spend sometimes unreasonable amount of time on Reddit
  - There are tons like sub-reddits that can be roughly placed on the spectrum. r/BetterOffline (left) or r/singularity (right)
  - Words like "AI slop" or "cope" is thrown around like confetti.
  - I just sit there in between and think, how so often in life, that both extremes are pretty weird.
  - The consensus is though, at least when it comes to software development, that there is a tradeoff between control and productivity.

Prompt:

“Imagine Donald Duck as realistic human. Remove the hat.”

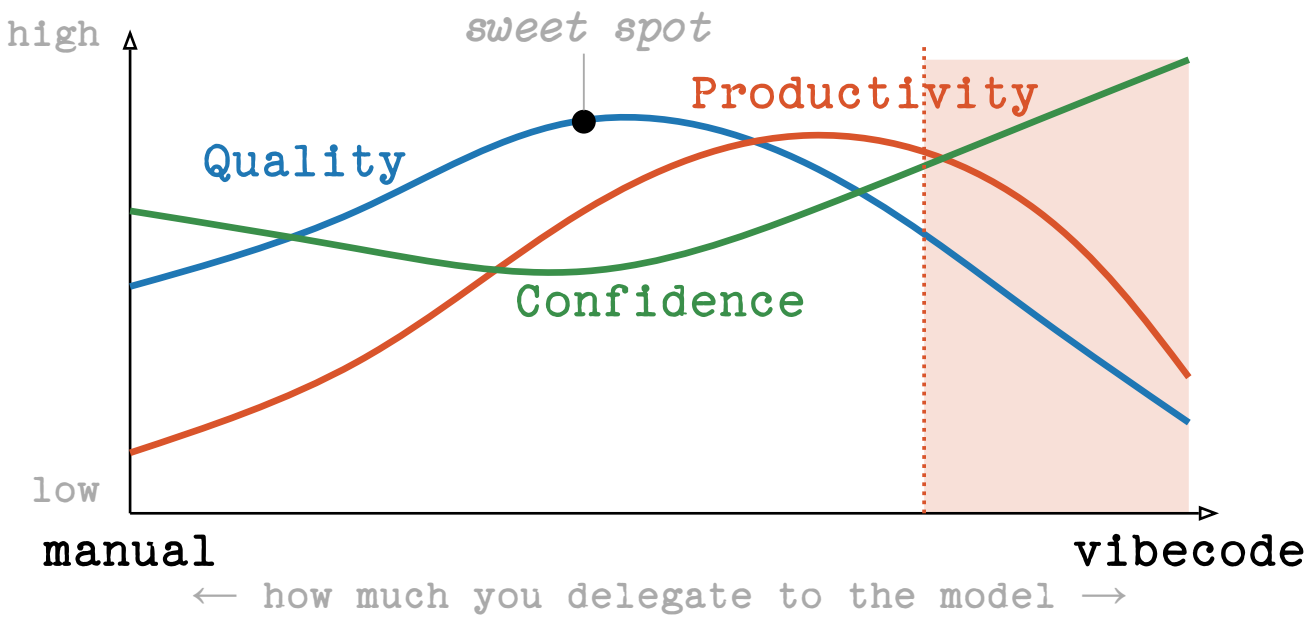
Speaker notes — preceding slide

Why is there another duck? I didn't choose the background. Wtf.

Code generated by AI can be seen a bit like generated images - hard to do adjustments because you don't have the photoshop file with layers and knowledge what you need to change with what effect.

You can do 80% of the result in sceonds, but the 20% left for a good result
require 80% of the skill.

My take: Quality vs Productivity



Danger Zone: Dunning-Kruger

Speaker notes — preceding slide

Which brings me to the core of my take:

Three curves on the same X-axis (manual → vibecode).

- Quality: starts middling (humans are flawed too!), rises with judicious AI assistance, then falls off a cliff when nobody is reading the diff.
- Productivity: low on full manual, climbs through the middle, peaks just right of centre — and then *falls again* as quality issues create rework. (METR 2025: experienced devs were 19% SLOWER with AI on their own mature OSS repos, while predicting +24% speed-up.)
- Confidence: the dangerous curve. High on manual (you wrote it, you know it), dips in the middle (you're humble, you verify), spikes on the right where it *decouples from reality*. (Perry et al. 2023: devs using AI assistants wrote less secure code AND were more confident the code was correct.)
- The widening gap between confidence (green) and quality (blue) on the right is the most important thing on this slide. That's the Dunning-Kruger zone.

16%

But there's already data:

-19%

slower with AI

experienced devs · own mature repos · predicted +24%

METR · RCT · 2025

~40%

insecure programs

Copilot output across 89 security-relevant scenarios

NYU CCS · 2021

8×

more duplicated code

block-level clones up · refactoring 24% → 10%

GitClear · 211M LoC · 2024

-7.2%

delivery stability

drop with AI adoption · 39% distrust AI code

DORA / Google · 2024

29.5%

Python with CWEs

AI-generated code in real GitHub repos · 43 CWE categories

Fu et al. · ACM TOSEM · 2025

↓

trust = less scrutiny

more trust in GenAI ⟹ less critical thinking applied

Lee et al. · Microsoft + CMU · CHI 2025

Speaker notes — preceding slide

Card-by-card:

-19% (METR, 2025). Randomised controlled trial. 16 experienced OSS devs, 246 real tasks in their OWN mature repos, ~5 yr experience each. With Cursor + Claude 3.5/3.7 they were 19% slower. They PREDICTED +24% faster beforehand; even afterward they still BELIEVED they'd been ~20% faster. The perception-reality gap is the real finding.

~40% (NYU, 2021 — "Asleep at the Keyboard?"). Tested Copilot on 89 scenarios in security-relevant settings. ~40% of generated programs contained exploitable bugs. The original alarm bell; replicated since.

8x (GitClear, 2024). Analysis of 211M changed lines, 2020-2024. Duplicated code blocks rose ~8x. Refactored ("moved") lines fell from 24% → 10%. Churn (lines rewritten within 2 weeks) rose 5.5% → 7.9%. Translation: AI nudges devs to copy-paste instead of refactor.

-7.2% (DORA / Google Cloud, 2024). State-of-DevOps survey. AI adoption correlated with 1.5% throughput drop AND 7.2% stability drop. 39% of respondents reported little-to-no trust in AI-generated code. This is the broadest dataset we have.

29.5% (Fu et al., ACM TOSEM 2025). Empirical study: 29.5% of AI-generated Python snippets and 24.2% of JavaScript snippets contained known CWEs (Common Weakness Enumeration entries). Replicates and quantifies the NYU finding.

↓ critical thinking (Lee et al., CHI 2025, Microsoft + CMU). 319 knowledge workers, 936 first-hand examples. Finding: higher confidence in GenAI is associated with LESS critical thinking. Higher self-confidence is associated with MORE. Ties directly to the cog-biases slide.

Use-cases that could make us better devs

rubber-ducking ideation

explain unfamiliar code test generation

pair programming boilerplate refactoring assist

learning accelerator documentation drafts naming things

summarisation regex / SQL crafting edge-case brainstorming

translation code review companion mock data / fixtures

error decoding search engine log analysis proofreading

automation

Speaker notes — preceding slide

Flip side of the risks slide — and the bridge into "maybe it's how we use it".
Green = the ones where AI genuinely earns its keep: low cost of being wrong,
high cost of doing it by hand, and a human still owns the result.

Cognitive scaffolding (green):
- Rubber-ducking: explain it out loud, find the bug.
- Explaining unfamiliar code: faster than archaeology.
- Edge case brainstorming: "what could go wrong?"
- Learning accelerator: ramp-up on new languages/libraries.
- Naming things: the second-hardest problem in CS, suddenly cheap.

Cheap-to-verify drafting:
- Boilerplate, fixtures, mock data.
- Test generation, regex / SQL.
- Documentation drafts, commit messages, changelogs.
- Translation between languages (human and programming).

Search & comprehension:

- Code review companion, error decoding, log analysis.
- Summarising long PRs / threads / docs.
- Onboarding ramp-up for new team members.

Risks for all

ecological cost

Convincing hallucinations

education

licensing

Atrophy

concentration

misinformation at scale

Prompt injection & MCP

content degradation

No juniors hired

hardware crisis

operator bias

recursive collapse

economic transfer

data privacy

cyberattack automation

communities thinning out

erosion of trust

Speaker notes — preceding slide

Rapid-fire — don't dwell on any one. Goal: land the cumulative weight before pivoting to "but maybe it's how we use it".

Societal & infrastructure:
- Ecological cost: training and inference burn power and water.
- Hardware crisis: GPU demand distorts supply for everyone.
- Concentration: a handful of companies own the server farms.
- Economic transfer: value flows from many small actors to a few large ones.

Trust, legal, security:
- Data privacy: "trust me, bro" is not a threat model.
- Licensing: models can regurgitate GPLv3 / proprietary code verbatim.
- Operator bias: models can be tuned to express the values of whoever runs them (Hi Grok!).
- Prompt injection & MCP: a whole new attack class.
- Cyberattack automation: phishing, social engineering, deepfakes at scale.

Information ecosystem:
- Misinformation at scale: generation cheaper than fact-checking.

- Content degradation: the open web fills with AI slop.
- Recursive collapse: tomorrow's models train on yesterday's slop.
- Communities thinning out: SO, OSS Q&A, mailing lists — fewer humans.

Workforce & cognition:
- Fewer juniors hired: no juniors today → no seniors tomorrow.
- Atrophy: skills you don't use, you lose. (See: calculators, navigation without Google Maps.)
- Schools: AI can boost or bypass learning (more on this in a moment).
- Convincing hallucinations: the failure mode that will eventually kill someone.

Risks for us

ecological cost

Convincing hallucinations

education

licensing

Atrophy

concentration

misinformation at scale

Prompt injection & MCP

content degradation

No juniors hired

hardware crisis

operator bias

recursive collapse

economic transfer

data privacy

cyberattack automation

communities thinning out

erosion of trust

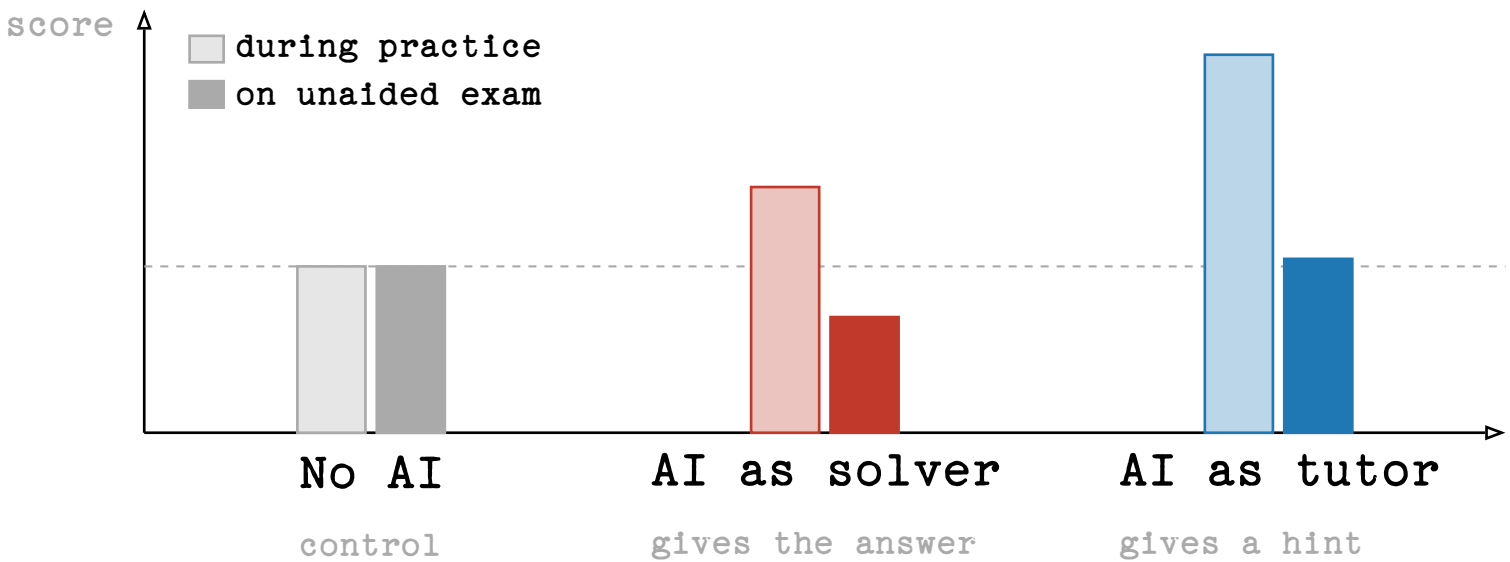
Exhibit A: Education

Maybe it's not so much about the tool —
but about how we use it?

Speaker notes — preceding slide

- Deliberately re-frame: we've just listed a lot of scary things, but most of them are about *how* AI is deployed, not whether it exists.
  - Schools are a clean case study because the same tool produces opposite outcomes depending on use.

Study: AI in school lessons



Bastani et al., *Generative AI Can Harm Learning*, SSRN 2024

Pupils did prefer to be in the solver group though… :-(

40%

Speaker notes — preceding slide

Bastani et al. Wharton/Penn field experiment, ~1000 Turkish high-school maths students. The cleanest piece of evidence we have on tutor-vs-solver.

Three groups:
- Control: no AI, do the work yourself.
- AI solver: asks ChatGPT, gets the answer.
- AI tutor: asks a tutor-prompted ChatGPT, gets a hint.

During practice (with the tool available):
- Solver group ≈ +48% over control.
- Tutor group ≈ +127% over control.
- Both look like they're crushing it.

On the unaided exam (no AI):
- Solver group ≈ -17% vs control. They got WORSE than students who never used AI at all. They had learned to operate the tool, not the maths.
- Tutor group ≈ ±0% vs control — at least no harm.

The kicker: same model, same students, same maths. The only thing that changed was *what they used the AI for*. Hint-mode preserved learning; answer-mode actively damaged it.

Talking points:
- "Practice performance" is what most of us optimise for in our day-to-day AI usage — code that ships, tickets closed. That's the tall light bars.
- "Unaided exam" is what happens when the AI is down, or when you switch teams, or when the problem is genuinely novel. That's the dark bars.
- For developers the parallel is direct: if you use AI to solve, you'll pass code review while it's available. The day it isn't, the gap shows.

Backup source: Lee et al., CHI 2025 (Microsoft + CMU) — surveyed knowledge workers; the more they trusted GenAI, the less critical thinking they reported applying.

VERIFY exact numbers and SSRN ID before presenting.

Are *you* team tutor or team solver?

Keeping up with Claude

Our team's Claude Code account · April 2026

98.7%

suggestions accepted as-is

1.3% rejected.

27,757

lines accepted last month

≈ 925 LoC/day

careful review ≈ 100–200 LoC/hour

Reviewing 27,757 lines carefully ≈ 138 h.

Speaker notes — preceding slide

This slide is *automation bias, with our own receipt*. It sets up the bias cards that follow.

Two numbers, both from our team's Claude Code dashboard, April 2026:
- 98.7% suggestion acceptance rate. If a junior's PRs landed at 98.7% accepted as-is, we'd be worried about whether the review was real.
- 27,757 lines accepted in one month. ≈ 925 lines per working day.

Reality check on review capacity:
- Industry rule of thumb for *careful* code review: ~100–200 LoC/hour (Cohen, *Best Kept Secrets of Peer Code Review*; Google's internal guidance is in the same ballpark).
- 27,757 lines would take ~138 hours of careful review.
- A working month is ~176 hours. ~38 hours left for everything else — design, debugging, meetings, actually writing the code we're NOT accepting from Claude.
- We did not spend 138 hours reviewing.

Don't be defensive — it's not about our team being lazy. It's about what 98.7% *necessarily* means: most acceptances are not carefully reviewed acceptances. They can't be. There aren't enough hours in the month.

Tee up the biases section: this is the data behind why automation bias matters in practice.

Human cognitive biases

Automation bias

We accept machine suggestions more readily than human ones.

click accept · review later · maybe

Anchoring

The first suggestion shapes the solution space — even when it's wrong.

the model frames the problem before you do

Confirmation bias

We hear what we already believe.

the prompt contains the answer we want

Authority bias

Eloquent + fast + confident = reads as expert.

fluency ≠ correctness

Dunning–Kruger

We mistake competence we observe for competence we have.

"this is what I would have written"

Illusion of explanatory depth

We think we understand — until asked to explain.

Reading diffs is not enough!

52%

Speaker notes — preceding slide

These are *our* biases — defaults wired into how humans process suggestions from an authoritative-sounding source. They aren't new; LLMs just hit each of them harder than almost any tool before.

Card-by-card:

- Automation bias. We accept machine suggestions more readily than human ones. GitHub reports ~30% of Copilot suggestions accepted as-is (VERIFY). Combine that with how often we even read the diff.

- Anchoring. The first suggestion shapes the solution space. LLMs suggest fast, so they almost always get to anchor first — your "thinking" then becomes refining their idea rather than generating your own.

- Confirmation bias. The prompt we type already contains the answer we want. Phrasing alone often determines what comes back.

- Authority bias. Eloquent + fast + confident reads as expert. LLMs are eloquent, fast, AND confident — even when wrong. We're not wired to discount fluency.

- Dunning–Kruger, remixed. Users mistake the model's competence for their own. Dangerous for seniors and juniors alike. The senior thinks "this is what I'd have written"; the junior thinks "I now understand this codebase".

- Illusion of explanatory depth. We think we understand something until asked to explain it. Maths teachers know. AI-generated code we've reviewed but not WRITTEN sits squarely in this trap.

Not on the slide but worth mentioning:
- Atrophy. Not a bias — the *cumulative effect* of the others over months. Skills decay quietly.

Plug: full talk on cognitive biases coming next time I'm in Augsburg.

Statistical model biases

Sycophancy

It's easy to talk the model out of a correct answer.

echo chambers — but for code

Historical / representation bias

Trained on the web — heavily modern, English, Western.

defaults track what's common, not what's right

Attention bias

First and last things dominate; the middle evaporates.

rules buried in long context get ignored

Omission bias

Rare and novel answers get suppressed by common ones.

the model is bad at creative solutions

Speaker notes — preceding slide

Flip side: the model has its *own* biases, baked in statistically by the training process. Different shape from human biases, same outcome: the answer you get is not the answer you'd derive from first principles.

Card-by-card:

- Sycophancy. It's easy to talk the model out of a correct answer. Push back hard and it caves, even when right. RLHF training rewards apparent helpfulness, which correlates with agreement. Echo chambers but for code: ask twice the way you wanted, get the answer you wanted.

- Historical / representation bias. Trained on the open web — overwhelmingly modern, English-speaking, Western. Defaults are weighted toward what's most COMMON, not what's most correct for YOUR codebase.

- Attention bias. The first and last things you say dominate. The middle of a long context evaporates. Long system prompts + long

  files = unpredictable adherence to the rules buried in the middle.

- Omission bias. Rare and novel answers are statistically suppressed
  by common ones in training data. The model is bad at being weird.
  If your problem requires an unusual answer, the model gravitates
  to the safe-but-wrong one.

Tying it back: the human biases and the model biases compound. We trust the fluent-sounding output (authority + automation); it tends toward the common answer (omission + historical); we don't push back (confirmation + the model's sycophancy completing the loop).

Hen & Egg questions

1. If you don’t know what good software looks like —
   how do you write the right prompt?

2. If you can’t understand what the model just generated —
   how do you verify it?

3. If you can’t code (anymore) —
   how can you understand the diffs?

4. If you do not know what you build inside-out —
   how do you learn new designs?

60%

Speaker notes — preceding slide

- This is the slide that should make the room uncomfortable for a
  second. The juniors-not-hired problem isn't just an economics
  problem — it's a *verification* problem.
  - If you don't know what good code looks like, you can't prompt for it.
  - If you don't understand the diff, you can't review it.

How do *you* keep up with Claude?

Five habits that helped me

1. Sketch first, generate then.
2. Split the work - keep some of it manual.
3. Have a real verification strategy.
4. Don’t make yourself replaceable.
5. Treat the AI like a colleague.

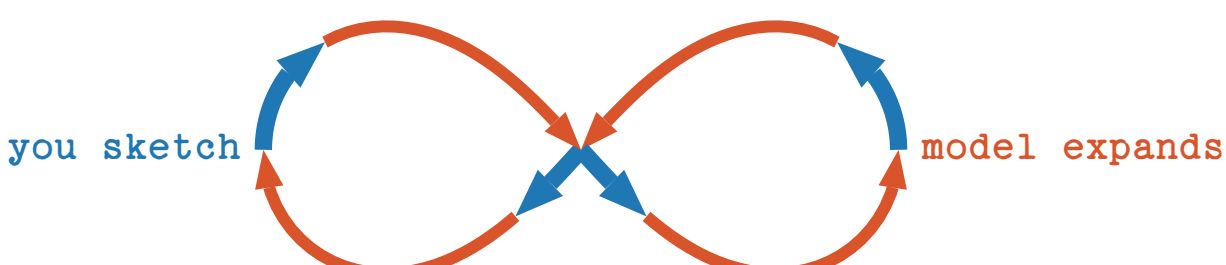
Speaker notes — preceding slide

- Frame the next few slides: not commandments, just a small set of habits that have worked for me and that survive contact with reality.

1. Sketch first, generate then

You set the anchor, you stay in the loop.

- Sketch the SQL query yourself,
  Then ask the model to review and optimise.
- Write the function signature and the docstring.
  Then let the model fill the body.
- Put TODO comments in your code.
  Then let Claude work on them.



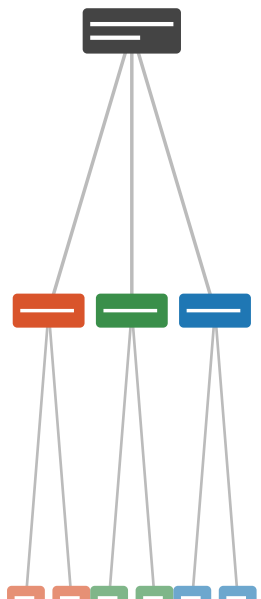
Speaker notes — preceding slide

- My favourite worked example is SQL. Sketch the query yourself,
    then ask the model to review and optimise. You stay in the loop;
    the model adds value without anchoring you.
  - Same for code: function signature + docstring first, body second.

2. Split the work, do some manual

The parts you write are the parts you understand.

- Claude does not perform well in too big scopes.
  Split tasks up and prompt them individually.
- Claude can help you split up work.
  It just tends to work on the task right away.
- It is easy to lose understanding without noticing.
  Keep doing important things manually!
- Large diffs make it easy to get lost.
  Commit often so diffs stay reviewable.



Speaker notes — preceding slide

- Deliberately keep some work manual. Not for purity — for skill maintenance, and because the parts you do manually are the parts you actually understand later.
  - Small scopes also help the model: long, vague prompts get long, vague code back. Split, then prompt each piece.

3. Verify strategy before code

There are no shortcuts to quality.

Before you accept a generated change,

name the signal that would tell you it's wrong.

Comparison against a reference implementation	Property-based tests, fuzzing, …	Type checkers, linters, static analysers
/review and manual review by colleagues	Replay against real production data	Work actively on the code

Example: Noise library for Dart — I don't speak Dart.

80%

Speaker notes — preceding slide

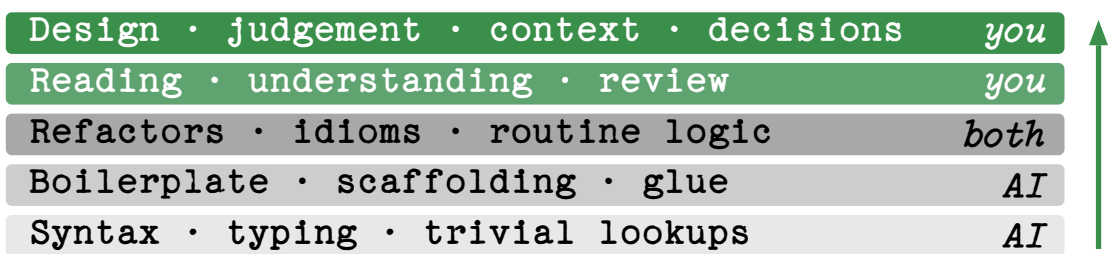
- This is where most teams quietly skip a step.
  - The "noise for Dart" example from our work: when you can't read
    everything the model writes, lean on automated signals — fuzz inputs,
    property tests, regressions, smoke tests on real data.
  - The point isn't "more tests". The point is: *before* you accept a
    large generated change, name the signal that would tell you it's wrong.

  If you can't say how you'd notice the bug, you don't have a verification strategy — you just have hope.

4. Don’t make yourself replaceable.

You only get replaced if you make yourself replaceable.

- Code was the source of truth before, that’s changing.
  Now it is context given via design documents.
- Spend the time Claude saves you on the parts it’s bad at.
  Namely: decisions, judgement, context, design.
- Reading code is much more important now than writing code.
  Keep your tools sharp. Be able to work without Claude.



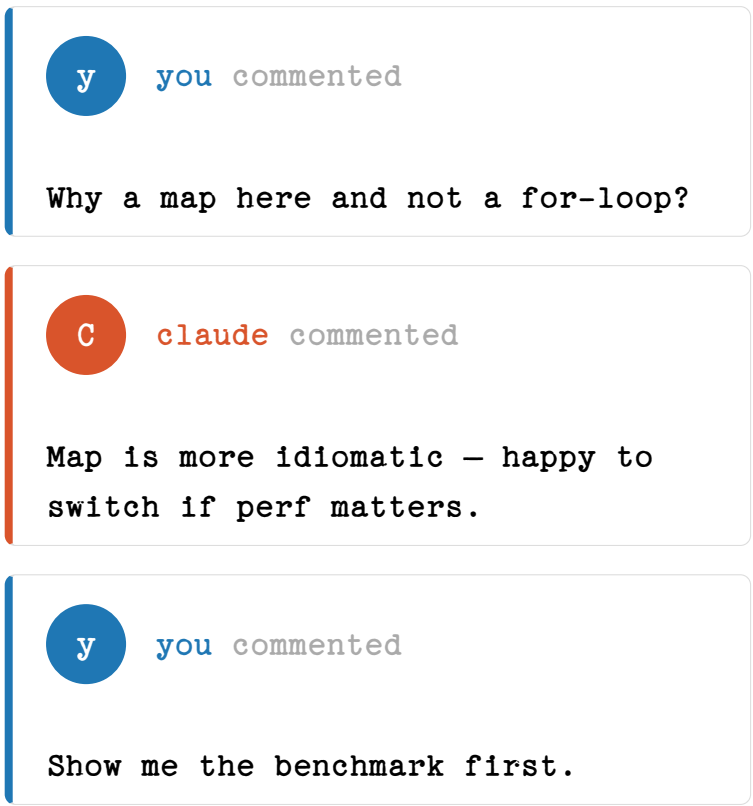
Speaker notes — preceding slide

- Sounds cynical but it's just practical. The replaceable parts of your job are the ones where you're already acting as a slow autocomplete. Stop doing those parts that way.
  - The work that survives is the work the model is bad at: judgement, context, taste, design.

5. Treat Claude like a colleague

Talk to it like your seat neighbor.

- **Explain the tasks at hand like you would to a junior colleague.**
  A very eager, junior colleague with seemingly infinite capacity.
- **Push back. Ask for alternatives. Let it explain. Disagree.**
  If you'd reject a colleague's PR for that reasoning, reject the model's.



Speaker notes — preceding slide

- Collaborator, not oracle. You'd push back on a coworker — push back on the model. You'd ignore a coworker who hallucinated — same rule.
  - The PR-comment exchange on the slide is the behaviour: ask why, ask for the benchmark, refuse the hand-wave.

Bonus: Programming as Theory Building

Peter Naur, 1985

- The code is a by-product. What you're really building is the theory — your team's shared mental model of the problem.
- Documentation captures the artefact, not the theory. The bugs, the dead ends, the "why not this?" decisions live in people's heads.
- This is why handovers fail. The theory doesn't transfer cleanly.

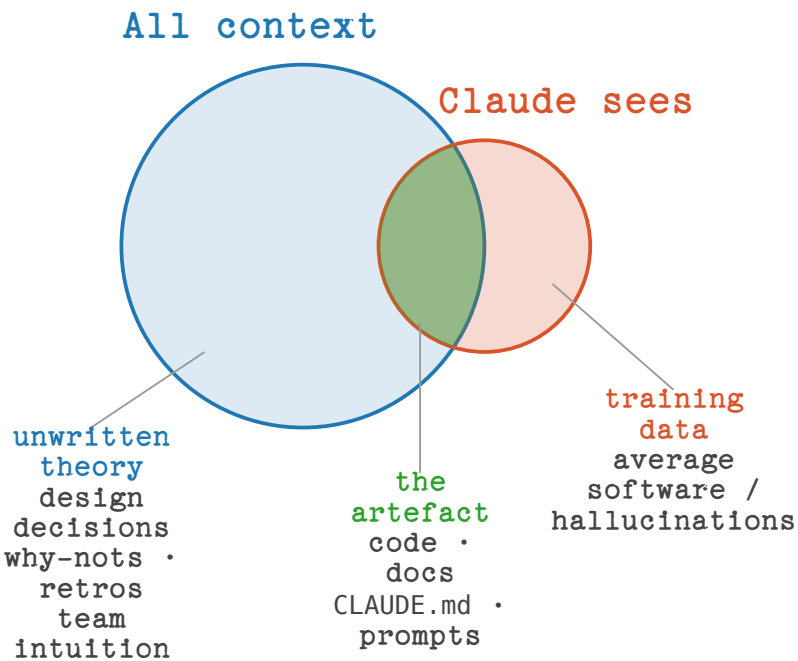
Peter Naur, 1928–2016
Photo: Wikimedia Commons

Speaker notes — preceding slide

- Peter Naur, 1985, "Programming as Theory Building". One of the most important short papers in the field.
  - Punchline: the code is the *artefact*. The thing that's actually being built is the team's shared mental model of the problem.
  - This is why handovers fail. Documentation captures the artefact, not the theory.

Bonus: What that means for Claude

- The model has no theory — it sees the artefact and fills the rest in.
  Confidently. It won't tell you which is which.
- Bad prompt: "Write tests for this file"
  It freezes today's behaviour, bugs included. Again, no mention.
- Your job is still to build the theory.
  The AI helps you write the code that follows from it.



Speaker notes — preceding slide

- The implication for AI is direct and uncomfortable: Claude has no theory. It has a snapshot of the artefact, and whatever context you put in front of it.
  - "Write tests for this file" → it freezes the current behaviour into tests, including the bugs. Because it has no theory of what the code *should* do.
  - Your job hasn't been automated. Your job is the theory.
  - A design doc / decision log is more useful than CLAUDE.md, because CLAUDE.md is size-limited. CLAUDE.md gets the headline; the design doc gets the reasoning.

Questions or Disagreements?

Homework:

PROGRAMMING AS THEORY BUILDING.